

Course: Parallel Algorithms

Topic: Deadlock Detection and Avoidance – Dining Philosophers Problem

Submission Method: Email submission

Submission Deadline: 12 November 2025

Objective

This assignment aims to deepen your understanding of *deadlock occurrence, detection, and avoidance* in concurrent systems using the classic *Dining Philosophers Problem*. You will demonstrate both a system that **falls into a deadlock** and **another that avoids a deadlock** using appropriate synchronization techniques.

Task 1: Deadlock Detection

Implement the *Dining Philosophers Problem* using multiple threads and shared resources (forks). Each philosopher alternates between thinking and eating. To eat, a philosopher must hold two forks (left and right). The amount of time each philosopher takes to think and eat is random. They can think for at most 10 seconds (or milliseconds) and they need at most 5 seconds (or milliseconds) for eating.

Your implementation should:

1. Allow the program to run **until a deadlock occurs**.
2. Each time a philosopher finishes eating, **print the philosopher's ID and the total number of times they have eaten**.
3. When a deadlock occurs, the **main thread must detect it** and print a clear message indicating that a deadlock has been detected.
4. Optionally, print a **summary table** showing how many times each philosopher has eaten before the deadlock.
5. **Bonus (optional):** Gracefully terminate the program once the deadlock is detected.

Expected Output Example:

```
Philosopher 1 finished eating (Total = 3)
Philosopher 3 finished eating (Total = 2)
Philosopher 5 finished eating (Total = 4)
...
*** DEADLOCK DETECTED ***
Summary:
P1 -> 3 times
P2 -> 2 times
P3 -> 4 times
P4 -> 1 time
P5 -> 3 times
```

Task 2: Deadlock Avoidance

Using one of the techniques discussed in class modify your solution so that **no deadlock can occur**.

Your code should still simulate the philosophers' actions (thinking, picking forks, eating) and print similar output showing the progress of eating.

Demonstrate that the program runs continuously without reaching a deadlock state.

Submission Requirements

- **Files:**
 - Source code for both parts (clearly labeled as *Deadlock Detection* and *Deadlock Avoidance*).
 - Console output (either screenshot or text file).
- **Include:** Your *name* and *student ID* in the email body and in your code comments.
- **Submission Method:** Send all files via email.

Grading Criteria

Component	Description	Marks
Correct implementation of the deadlock scenario	Working code that produces a deadlock and detects it	40%
Correct implementation of deadlock avoidance	Code runs indefinitely without a deadlock	40%
Output readability and reporting	Meaningful and clear outputs with a summary	10%

Component	Description	Marks
Bonus	Graceful exit upon deadlock detection	+5%
Code quality and comments	Proper documentation and style	10%